

* Always Remember: → Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. — vHixem

* Problem: —

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position. Return true if you can reach the last index or false otherwise.

Input: `nums = [2, 3, 1, 1, 4]`

Output: true

↪ Top

Input: `nums = [3, 2, 1, 0, 4]`

Output: false

→ If array size is n then top is $n-1$.

* Prerequisites (Very Important):-

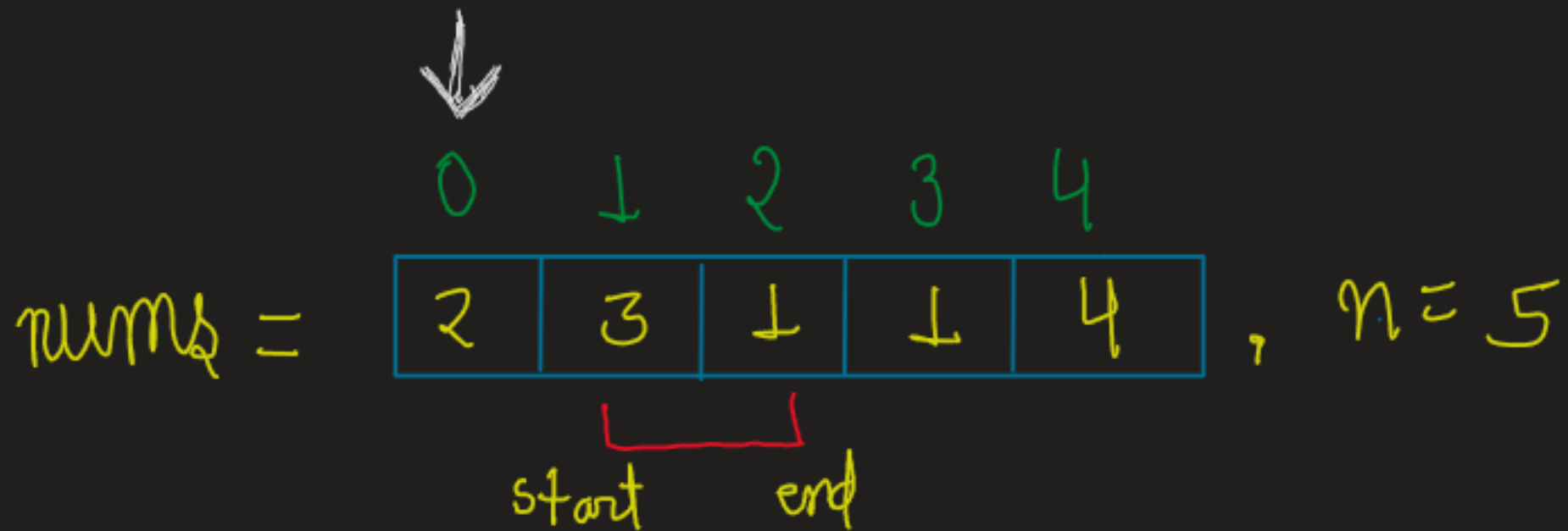
→ This question involves to explore multiple options / possibilities, means number of options are not fixed.

→ So Loops are used in such questions, To identify how to use Loops in such questions, Watch Lecture 10 - 1D DP and then comeback here.

→ In that, I have provided deep view on how to identify loops.

* Jump Range of any Index :-

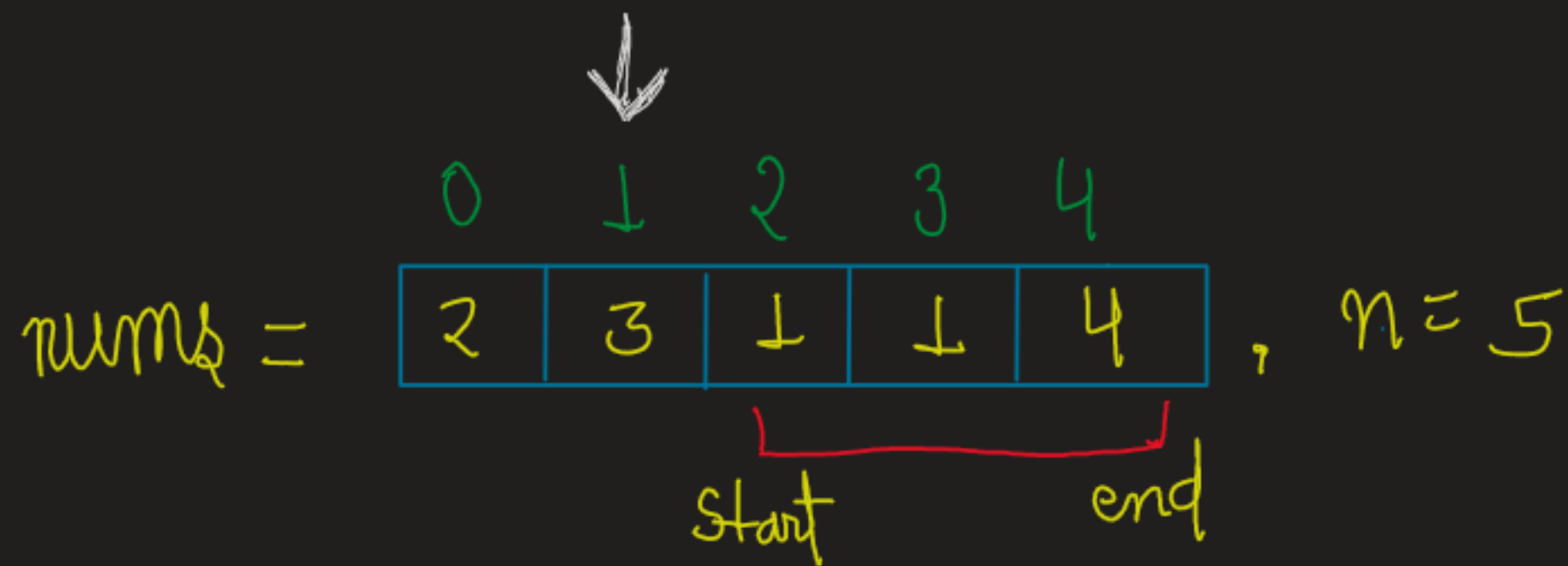
→ Jump options from $i=0$



~ here we have 2 jump options

$i \rightarrow i+1$
 $i \rightarrow i+2$

→ Jump options from $i = 1$



~ here we have 3 jump options

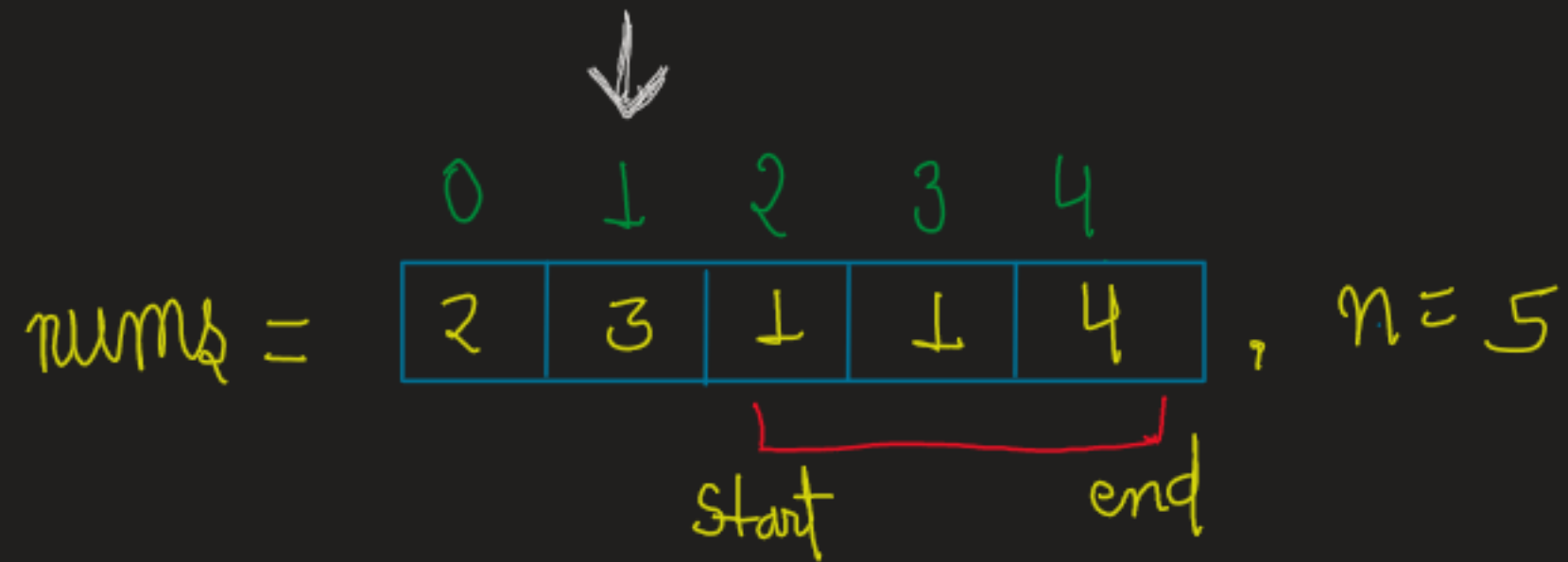
i to $i+1$

i to $i+2$

i to $i+3$

⇒ So $nums[i]$ represents Jump options

→ So $\text{nums}[i]$ represents Jump options



$$i = 1, \text{ start} = i + 1$$
$$\text{end} = i + \text{nums}[i]$$

→ So Jump options are not fixed, could vary right.

→ That's why we will run a loop to explore all options.

↓

0	1	2	3	4
2	3	1	1	4

start end

nums = , n = 5

i = 1, start = i + 1
end = i + nums[i]

```

for (int j = start; j <= end; j++) {
    // Jump from i to jth index
    // we want to find can reach index n-1
    // so call Recursion(j), do the same
    bool canReachEndFrom-j = Recursion(j);
    if (canReachEndFrom-j) return true;
}

```